

E-Commerce Platform

Complete Database Management System Project

Submitted in partial fulfilment of the requirements for the degree of B.Tech / BCA / MCA

Phase 1: DB Design & ER | Phase 2: Relational Algebra & SQL | Phase 3: Normalization

Submitted by

Palak Ramtri — 2410031494
Priyam Bhowmik — 2410031539
Harsh Aggarwal — 2410031541
Anushka Rajput — 2410031519

PHASE 1: DATABASE DESIGN & ER DIAGRAM

1. Introduction

This project designs and implements a relational database for an E-Commerce Platform using MySQL. The database covers user management, product catalogues, shopping cart, order lifecycle, payment processing, and product reviews — applying core DBMS concepts throughout.

1.1 Objectives

- Design a normalised relational schema for a real-world e-commerce system
- Implement tables with PK, FK, CHECK, UNIQUE, and ENUM constraints
- Write SQL queries covering joins, aggregations, subqueries, and window functions
- Apply Relational Algebra notation to express query logic formally
- Demonstrate normalisation through 1NF, 2NF, 3NF, and BCNF analysis
- Implement COMMIT/ROLLBACK transactions and concurrency control

1.2 Technology Stack

- Database Engine: MySQL 8.0+
- Query Language: SQL (DDL, DML, DCL, TCL)
- Tools: MySQL Workbench / phpMyAdmin
- Storage Engine: InnoDB (ACID-compliant, row-level locking)

2. Entity-Relationship Model

2.1 Entities and Attributes

Entity	Primary Key	Key Attributes	Role
Users	user_id	name, email, password_hash, role	Stores all platform users
Sellers	seller_id	user_id(FK), shop_name, gstin, rating	Extends users for sellers
Addresses	address_id	user_id(FK), street, city, state, pincode	Delivery addresses per user
Categories	category_id	parent_id(FK), name, description	Hierarchical product taxonomy
Products	product_id	seller_id(FK), category_id(FK), price, stock_qty	Product listings
Cart	cart_id	user_id(FK), product_id(FK), quantity	Temporary pre-order cart
Orders	order_id	user_id(FK), address_id(FK), status, total_amount	Placed orders
Order Items	order_item_id	order_id(FK), product_id(FK), quantity, unit_price	Order line items
Payments	payment_id	order_id(FK), amount, method, status	One payment per order
Reviews	review_id	user_id(FK), product_id(FK), rating, comment	Product reviews

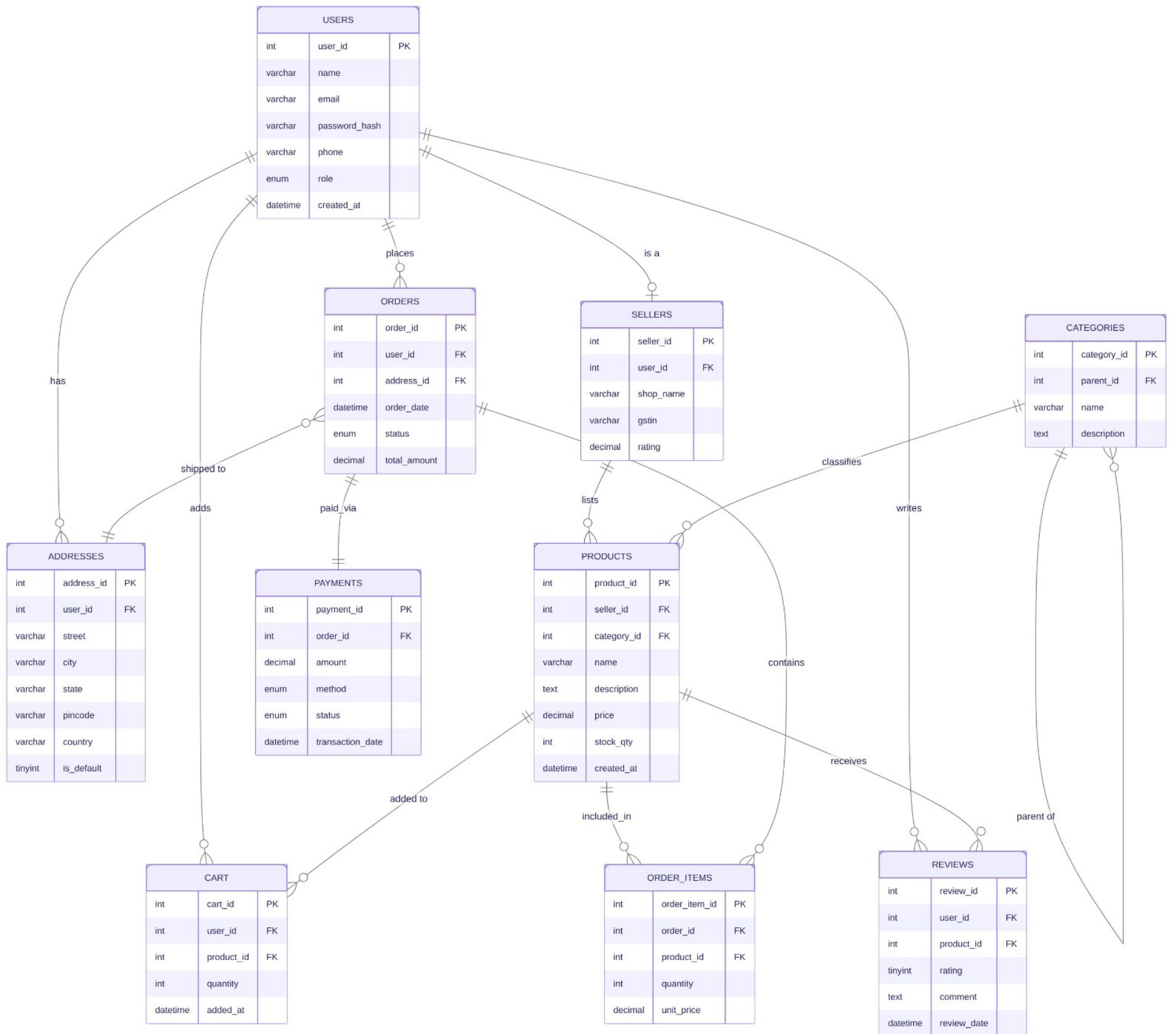
2.2 Relationships

From	To	Cardinality	Description
Users	Addresses	1 : M	A user can have multiple delivery addresses
Users	Sellers	1 : 1	A user account can be linked to one seller profile
Users	Orders	1 : M	A customer places many orders over time
Users	Reviews	1 : M	A user can write many product

From	To	Cardinality	Description
			reviews
Sellers	Products	1 : M	A seller lists many products
Categories	Products	1 : M	A category contains many products
Categories	Categories	Self-ref	Parent – child hierarchy for subcategories
Orders	Order Items	1 : M	Each order contains multiple line items
Orders	Payments	1 : 1	Each order has exactly one payment record
Orders	Addresses	M : 1	Many orders can ship to the same address
Products	Order Items	1 : M	A product can appear in many order lines
Products	Reviews	1 : M	A product can have many reviews

2.3 Key Design Decisions

- role ENUM on users table distinguishes customers, sellers, admins without separate tables.
- Self-referential FK on categories.parent_id enables unlimited category nesting.
- UNIQUE(user_id, product_id) on cart prevents duplicate entries for same product.
- UNIQUE(order_id) on payments enforces strict one-to-one relationship.
- unit_price stored in order_items (not fetched from products) preserves historical price.
- DECIMAL(10,2) used for all monetary columns to prevent floating-point errors.
- InnoDB engine used throughout for FK enforcement and transaction



PHASE 2: RELATIONAL ALGEBRA & SQL QUERIES

3. Relational Algebra

Relational Algebra provides the formal mathematical foundation for SQL. The following operators are used:

Operator	Symbol	Meaning
Selection	σ (sigma)	Filters rows based on a predicate condition
Projection	π (pi)	Selects specific columns; eliminates duplicates
Natural Join	$\bowtie$	Combines tables on matching column names/values

Operator	Symbol	Meaning
Union	U	Combines rows from two compatible relations
Set Difference	–	Rows in first relation but not in second
Intersection	∩	Rows common to both relations
Cartesian Product	×	All combinations of tuples from two relations
Rename	ρ (rho)	Assigns a new name to a relation or attribute
Aggregate	γ (gamma)	Grouping with aggregate functions (SUM, AVG, COUNT)
Division	÷	Finds tuples in R associated with all tuples in S

3.1 Notable Relational Algebra Expressions

RA-1: All Customer Names and Emails

Expression: $\pi \text{ user_id, name, email } (\sigma \text{ role='customer' (users)})$

```
SELECT user_id, name, email
FROM users WHERE role = 'customer';
```

RA-8: Customers with No Orders (Anti-Join)

Expression: $\pi \text{ user_id, name, email } (\sigma \text{ role='customer' (users)}) - \pi \text{ user_id (users} \bowtie \text{orders)}$

```
SELECT u.user_id, u.name, u.email
FROM users u
LEFT JOIN orders o ON u.user_id = o.user_id
WHERE o.order_id IS NULL AND u.role = 'customer';
```

RA-12: Subcategories (Self-Join)

Expression: $\pi \text{ parent.name, child.name } (\rho \text{ parent(categories)} \bowtie \rho \text{ child(categories)})$

```
SELECT c1.name AS parent, c2.name AS sub_category
FROM categories c1 JOIN categories c2 ON c2.parent_id = c1.category_id
WHERE c1.name = 'Electronics';
```

RA-4: Revenue Per Seller

Expression: $\gamma \text{ seller_id; SUM(quantity} \times \text{unit_price)} \rightarrow \text{revenue } (\sigma \text{ status} \notin \{\text{cancelled, returned}\} (\text{order_items} \bowtie \text{products} \bowtie \text{sellers} \bowtie \text{orders}))$

```
SELECT s.shop_name,
       COUNT(DISTINCT o.order_id) AS total_orders,
       SUM(oi.quantity * oi.unit_price) AS total_revenue
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
```



```

JOIN sellers s ON p.seller_id = s.seller_id
JOIN orders o ON oi.order_id = o.order_id
WHERE o.status NOT IN ('cancelled','returned')
GROUP BY s.seller_id ORDER BY total_revenue DESC;

```

3.2 All 20 SQL Queries Summary

#	Query Description	Concept
1	List all products with category and seller	3-table JOIN, ORDER BY
2	All orders with customer name and status	JOIN, ORDER BY
3	Order details — items inside each order	Multi-table JOIN, computed column
4	Total revenue per seller	GROUP BY, SUM, COUNT DISTINCT
5	Average product rating	AVG, COUNT, GROUP BY
6	Cart contents for specific user (user_id=1)	WHERE filter, computed total
7	Products with low stock (qty < 50)	WHERE, ORDER BY
8	Customers who never placed an order	LEFT JOIN, IS NULL (anti-join)
9	Most popular products by units sold	SUM, GROUP BY, LIMIT
10	Payment summary by method and status	GROUP BY, COUNT, SUM
11	Monthly revenue trend	DATE_FORMAT, GROUP BY month
12	Subcategories of Electronics	Self-JOIN on categories
13	Top 5 customers by spend	SUM, ORDER BY, LIMIT
14	Products reviewed but never ordered	Subquery with NOT IN
15	Orders with full delivery address	JOIN, CONCAT
16	All user emails — customers and sellers	UNION
17	Customers with at least one delivered order	EXISTS correlated subquery
18	Products priced above their category average	Correlated subquery
19	Top-selling product per seller	CTE + RANK() window function
20	Running total of successful payments	SUM() OVER window function

PHASE 3: NORMALIZATION OF DATABASE

4. Normalization

Database normalization is the process of structuring a relational database to reduce data

redundancy and improve data integrity. We apply normalization progressively from UNF through BCNF.

4.1 Unnormalised Form (UNF)

Before normalization, all data might be stored in one flat table:

```
OrderFlat(order_id, customer_name, customer_email, customer_phone,  
          product1_name, product1_price, product2_name, product2_price,  
          seller_shop, payment_method, delivery_city)
```

Problems: Repeating groups (product1, product2...), multi-valued attributes, massive redundancy.

4.2 First Normal Form (1NF)

Rule: All attributes must be atomic; no repeating groups or multi-valued attributes. Each cell must contain a single value.

- Violation: Storing multiple products in one row (product1, product2...).
- Fix: Each product becomes its own row in order_items.
- All columns in our schema hold exactly one value per cell. ✓

Our tables satisfy 1NF because every column is atomic and there are no repeating groups.

4.3 Second Normal Form (2NF)

Rule: Must be in 1NF + every non-key attribute must be fully functionally dependent on the entire primary key (no partial dependencies).

- Applies to tables with composite primary keys.
- Example violation: if cart stored product_name, that would depend only on product_id — not the full key (user_id, product_id).
- Fix: product_name stays in products; cart only stores product_id (FK). ✓
- order_items stores unit_price (price at time of purchase) — fully dependent on order_item_id. ✓

All our tables satisfy 2NF as no partial dependencies exist.

4.4 Third Normal Form (3NF)

Rule: Must be in 2NF + no transitive dependencies (non-key attributes must not depend on other non-key attributes).

- Example violation: if products stored shop_name, then: product_id → seller_id → shop_name (transitive).
- Fix: shop_name moved to sellers table; products only stores seller_id (FK). ✓
- Similarly, category_name stays in categories; products stores category_id only. ✓

All our tables satisfy 3NF — no transitive dependencies remain.

4.5 Boyce-Codd Normal Form (BCNF)

Rule: For every non-trivial functional dependency $X \rightarrow Y$, X must be a superkey.

- reviews table: composite key is (user_id, product_id).
- Every FD: review_id $\rightarrow$ all attributes (PK), (user_id,product_id) $\rightarrow$ rating, comment (composite key). ✓
- No non-key attribute determines another. ✓

Our schema satisfies BCNF. No further decomposition is required.

4.6 Functional Dependencies Summary

Table	Functional Dependencies	Normal Form
users	user_id $\rightarrow$ name, email, password_hash, phone, role, created_at	BCNF ✓
sellers	seller_id $\rightarrow$ user_id, shop_name, gstin, rating	BCNF ✓
addresses	address_id $\rightarrow$ user_id, street, city, state, pincode, country	BCNF ✓
categories	category_id $\rightarrow$ parent_id, name, description	BCNF ✓
products	product_id $\rightarrow$ seller_id, category_id, name, price, stock_qty	BCNF ✓
cart	(user_id,product_id) $\rightarrow$ quantity, added_at	BCNF ✓
orders	order_id $\rightarrow$ user_id, address_id, order_date, status, total_amount	BCNF ✓
order_items	order_item_id $\rightarrow$ order_id, product_id, quantity, unit_price	BCNF ✓
payments	payment_id $\rightarrow$ order_id, amount, method, status, transaction_date	BCNF ✓
reviews	(user_id,product_id) $\rightarrow$ rating, comment, review_date	BCNF ✓

Future Scope

- Coupons & discount engine with complex pricing rules
- Wishlist functionality with share and comparison features
- Seller analytics dashboard with real-time revenue charts
- Recommendation engine using collaborative filtering

- Shipping tracking integration with third-party APIs
- Redis caching layer for high-traffic product pages
- Read replicas for horizontal scaling of SELECT queries
- Full-text search on product descriptions using MySQL FULLTEXT indexes

References

- Ramakrishnan, R. & Gehrke, J. — Database Management Systems, 3rd Ed., McGraw-Hill
- Silberschatz, A., Korth, H., Sudarshan, S. — Database System Concepts, 7th Ed.
- Date, C. J. — An Introduction to Database Systems, 8th Ed., Addison-Wesley
- MySQL 8.0 Official Documentation — <https://dev.mysql.com/doc/refman/8.0/en/>
- Elmasri, R. & Navathe, S. — Fundamentals of Database Systems, 7th Ed.